

Alumno: Diego Mora Márquez
Profesor: Luis de Jesús López Hernandez

Semestre 4

Mérida Yucatán **2026**

Implementación de Shader en el Proyecto

Shader de Agua Dinámica para Cenote · Universal Render Pipeline · Proyecto Cenote Vivo

1. Shader implementado

El elemento visual más importante de *Cenote Vivo* es el agua: es lo que el usuario ve cambiar cuando simula la contaminación y es el símbolo de lo que el proyecto quiere proteger. Por eso se desarrolló un shader específico, **`CenoteVivo/CenoteWater`**, para URP. Reproduce cuatro fenómenos del agua de un cenote real —la transparencia que deja ver el fondo, el color que se intensifica con la profundidad, el reflejo del cielo en ángulos rasantes y el movimiento de la superficie— y permite degradarlos de forma continua con un solo parámetro, `_PollutionLevel`.

El shader se escribió en **HLSL sobre las librerías de URP** y no en Shader Graph. Shader Graph genera internamente un código equivalente, pero escribirlo a mano dio control directo sobre la precisión de cada variable (`half` frente a `float`, relevante en GPU móviles), permitió dibujar la cara inferior del agua para la vista sumergida con `SV_IsFrontFace` y evitó las variantes y pasadas que Shader Graph añade por defecto. Para facilitar la lectura, la Figura 1 presenta el mismo cálculo como un grafo de nodos de Shader Graph; cada nodo corresponde a un bloque del código.

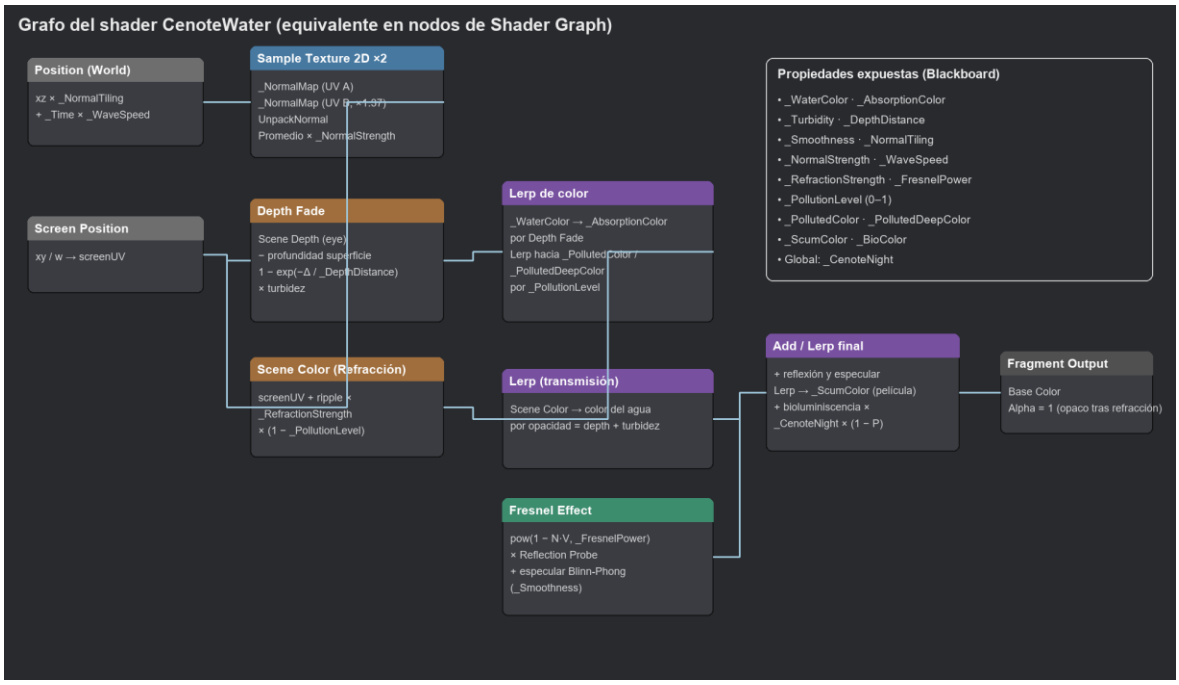


Figura 1. Árbol de nodos equivalente del shader CenoteWater: Depth Fade, Fresnel, distorsión por mapa de normales y mezcla de colores.

2. Propiedades expuestas

Tabla 1. Propiedades del material M_CenoteAgua.

Propiedad	Tipo	Valor por defecto	Función
`_WaterColor`	Color	(0.10, 0.78, 0.80)	Turquesa del agua somera.
`_AbsorptionColor`	Color	(0.01, 0.20, 0.32)	Color al que tiende el agua profunda.
`_Turbidity`	Rango 0–1	0.05	Opacidad base; la escribe el simulador.
`_DepthDistance`	Float	6 m	Profundidad a la que el agua limpia se vuelve casi opaca.
`_Smoothness`	Rango 0–1	0.92	Nitidez del brillo especular y del reflejo.
`_NormalMap`, `_NormalTiling`, `_NormalStrength`	Textura / Float	0.18 · 0.55	Ondas de la superficie.
`_WaveSpeed`, `_RefractionStrength`, `_FresnelPower`	Float	0.035 · 0.025 · 4	Movimiento, distorsión y reflejo.
`_PollutionLevel`	Rango 0–1	0	**Parámetro principal:** de agua limpia a contaminada.
`_PollutedColor`, `_PollutedDeepColor`, `_ScumColor`	Color	Marrón / casi negro / verde oliva	Colores del agua contaminada y de la película de residuos.
`_BioColor` + global `_CenoteNight`	Color / Float	Cian · 0	Bioluminiscencia nocturna.

3. Funcionamiento por etapas

3.1 Distorsión por mapa de normales

En el vertex shader se calculan dos juegos de coordenadas a partir de la posición en el mundo (`xz · _NormalTiling`), desplazados en sentidos opuestos con el tiempo y uno de ellos con una escala de 1.37 para que los patrones no se repitan al unísono. El fragment shader muestrea **dos veces el mismo mapa de normales** y promedia el resultado. Calcular las coordenadas por vértice ahorra operaciones en cada píxel, y reutilizar una sola textura evita cargar dos mapas en memoria. La normal resultante perturba la refracción, el reflejo y el brillo.

3.2 Depth Fade (absorción por profundidad)

El shader lee el *depth buffer* de la escena (`_CameraDepthTexture`) en el píxel actual, lo convierte a distancia lineal y le resta la distancia de la superficie del agua. La diferencia es el grosor de la columna de agua que atraviesa la mirada. Siguiendo la ley de Beer–Lambert, la opacidad crece de forma exponencial con ese grosor:

$$\text{depthFade} = 1 - \exp(-\text{grosor} / (_\text{DepthDistance} \cdot \text{lerp}(1, 0.06, \text{turbidez})))$$

Con el agua limpia se necesitan varios metros para que el agua se vuelva opaca; con la turbidez al máximo bastan unos 36 cm. El color se interpola de `_WaterColor` a `_AbsorptionColor` según este factor.

3.3 Refracción ligera

En lugar de trazar rayos refractados, el shader toma la imagen de la escena opaca que URP copia antes de dibujar los transparentes (`_CameraOpaqueTexture`) y la muestrea con un pequeño desplazamiento dado por las ondas. El efecto se atenúa en las orillas (donde el grosor es pequeño) para no arrastrar objetos que están fuera del agua, y se multiplica por (1 − contaminación): el agua sucia deja de mostrar el fondo. La copia opaca se configuró a la mitad de resolución con filtrado bilineal, suficiente para un fondo que de todos modos se ve distorsionado.

3.4 Fresnel, reflexión y especular

El efecto Fresnel `pow(1 - N·V, _FresnelPower)` hace que el agua refleje poco cuando se mira desde arriba y mucho en ángulos rasantes, que es lo que ocurre en la realidad. El reflejo proviene del *reflection probe* de la escena (`GlossyEnvironmentReflection`) y el brillo del sol se calcula con un término Blinn-Phong cuyo exponente depende de la suavidad. Al contaminarse, la suavidad baja un 65 % y la intensidad del reflejo un 60 %: el agua se ve mate.

3.5 Mezcla final, contaminación y noche

Todos los colores del agua limpia se interpolan hacia sus equivalentes contaminados con `_PollutionLevel`. Por encima del 35 % aparece una película de residuos, que se obtiene sin texturas extra comparando los dos muestreos del mapa de normales. Finalmente se suma la bioluminiscencia: destellos cian que dependen de la variable global `_CenoteNight` y que se multiplican por (1 − contaminación). El shader devuelve un color opaco con `Blend One Zero`, porque la transparencia ya está resuelta con la imagen refractada; así se evita la mezcla alfa, que en GPU móviles obliga a leer el framebuffer.

Fragmento central del fragment shader (CenoteWater.shader).

```
half turbidity = lerp(_Turbidity, 1.0h, P);
float distanceScale = _DepthDistance * lerp(1.0, 0.06, turbidity);
half depthFade = (half)(1.0 - exp(-thickness / distanceScale));

half3 shallow = lerp(_WaterColor.rgb, _PollutedColor.rgb, P);
half3 deep    = lerp(_AbsorptionColor.rgb, _PollutedDeepColor.rgb, P);
half3 waterCol = lerp(shallow, deep, depthFade);

float2 refrUV = screenUV + ripple * _RefractionStrength * (1.0h - P) * saturate(thickness);
half3 behind  = SampleSceneColor(refrUV) * shallow * 1.6h;
half3 col     = lerp(behind, waterCol, saturate(depthFade + turbidity * 0.9h));

half fresnel = pow(1.0h - saturate(dot(N, V)), _FresnelPower);
half3 refl   = GlossyEnvironmentReflection(reflect(-V, N), 1.0h - smooth, 1.0h);
col += (refl * fresnel * (1.0h - 0.6h * P) + spec) * (isFrontFace ? 1.0h : 0.25h);
```

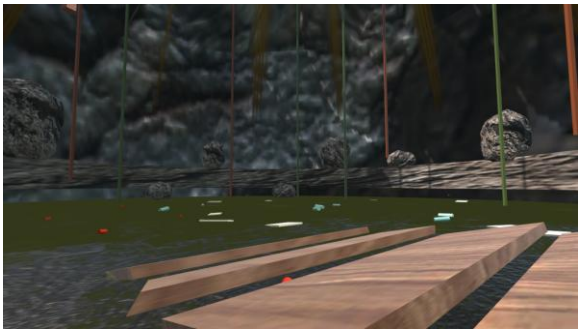
4. Evidencia visual



_PollutionLevel = 0



_PollutionLevel = 0.5



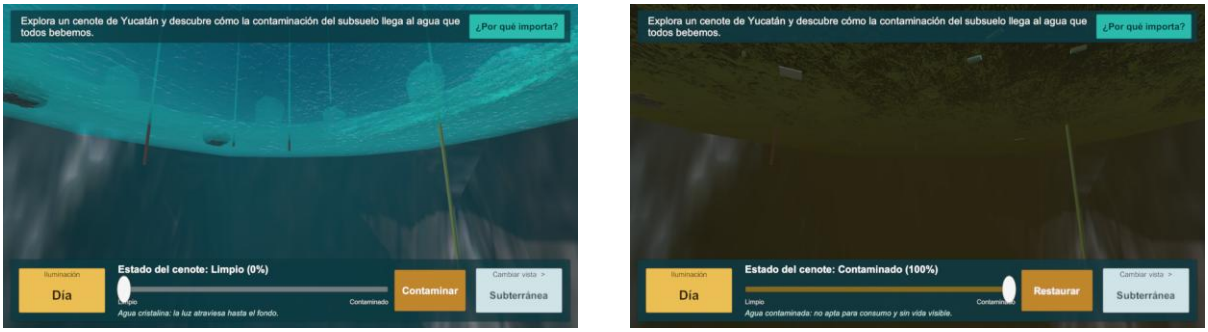
_PollutionLevel = 1



Noche: bioluminiscencia

Figura 2. El mismo material con distintos valores del parámetro de contaminación, y de noche con el agua limpia.

Con el agua limpia se distinguen el turquesa somero, la distorsión de los tablones sumergidos y los reflejos del entorno en la parte lejana, donde el ángulo es rasante. A la mitad del rango el agua ya no deja ver el fondo y su color se vuelve verdoso; al máximo es una superficie opaca y mate, con manchas de residuos. De noche, solo el agua limpia muestra destellos.



Vista sumergida, limpia **Vista sumergida, contaminada**

Figura 3. La cara inferior de la superficie, visible desde la Vista 3 gracias a Cull Off y SV_IsFrontFace.

5. Justificación técnica

5.1 Por qué no usar el shader Lit o Standard

El material original del agua usaba *Universal Render Pipeline/Lit* con una textura de agua. Ese shader sirve para superficies sólidas, pero tiene dos carencias frente a lo que necesita el proyecto:

- **Falta de control sobre la profundidad visual.** Lit no conoce la escena que hay detrás de la superficie: su transparencia es un alfa constante, igual en la orilla que en el centro del cenote. No puede simular la absorción por profundidad ni la refracción, que son justamente las pistas visuales que distinguen un agua cristalina de una turbia.
- **Costo de sombreado innecesario.** Lit evalúa el modelo PBR completo: GGX, varias luces adicionales, oclusión, emisión, mapas de detalle y múltiples variantes de *keywords*. El agua del cenote solo necesita una luz principal, un reflejo de entorno y un término especular sencillo. En modo transparente, además, Lit hace mezcla alfa sobre el framebuffer, que es una operación costosa en arquitecturas móviles.

5.2 Cómo resuelve el objetivo del proyecto

El shader convierte la contaminación en un **parámetro continuo**. `PollutionSimulator.cs` escribe `_PollutionLevel` y `_Turbidity` en el material cada vez que el nivel cambia, y el shader interpola de forma coherente color, opacidad, refracción, brillo y movimiento. No hay dos materiales que se

intercambien: el usuario ve una sola agua que se degrada poco a poco, lo que comunica mejor la idea de que la contaminación del subsuelo es un proceso acumulativo. La relación con la noche —la bioluminiscencia desaparece en el agua contaminada— añade una segunda lectura del mismo mensaje: la contaminación apaga la vida.

5.3 Optimización para móviles

- **Operaciones simples:** todo el fragment shader se compone de interpolaciones, productos punto, una exponencial y una potencia. No hay bucles, ramificaciones dependientes de datos ni muestreos en cadena.
- **Pocas lecturas de textura:** dos del mapa de normales (la misma textura), una de profundidad y una de la imagen opaca; cuatro en total.
- **Precisión media (`half`)** en colores, normales y factores, que en Mali y Adreno se ejecuta al doble de velocidad y con menos registros que `float`. Solo las coordenadas y las profundidades usan `float`.
- **Trabajo trasladado al vertex shader:** las coordenadas animadas se calculan por vértice; la malla del agua tiene 200 triángulos frente a cientos de miles de píxeles.
- **Empaquetado eficiente de canales:** el mismo mapa de normales alimenta las ondas (canales XY), la película de residuos (canal Z) y la bioluminiscencia (diferencia de canales), sin texturas adicionales.
- **Sin mezcla alfa y compatible con SRP Batcher:** las propiedades del material están en el `CBUFFER UnityPerMaterial`, lo que permite que URP agrupe el draw call, y la salida opaca evita leer el framebuffer.
- **Texturas comprimidas en ASTC 4×4** para el mapa de normales en Android, con mipmaps y un tamaño máximo de 512 px.

Los shaders complementarios siguen la misma filosofía: `CenoteSeep` dibuja las filtraciones sin texturas, con un ruido calculado a partir de las coordenadas y una única variable global para todas las franjas; y `ParticleSoft` genera la forma circular de las partículas dentro del shader, sin textura, con alfa premultiplicado para servir tanto a mezcla normal como a mezcla aditiva.

6. Conclusión

Un shader hecho a la medida del problema permitió que el elemento central de la aplicación fuera a la vez más expresivo y más barato que el material genérico. La absorción por profundidad, la

refracción y el Fresnel dan al agua la lectura de «agua de cenote», y el parámetro `_PollutionLevel` traduce el concepto abstracto de contaminación del subsuelo en un cambio visual inmediato y gradual que el usuario controla con un dedo.